Shortest path algorithm

Michel Bierlaire

Practice quiz

The shortest path algorithm, to calculate the shortest paths from node o to all the other nodes in the network, is described as follows.

1. Initialization: $\mathcal{S} \leftarrow \{o\}$, $\lambda_o = 0$, $\lambda_j = +\infty$.
2. At each iteration:
   - Select i in $\mathcal{S}$ and remove it from $\mathcal{S}$.
   - For each (i, j):
     - If $\lambda_j > \lambda_i + c_{ij}$,
       * $\lambda_j \leftarrow \lambda_i + c_{ij}$.
       * If $\lambda_j < 0$ and $\lambda_j < (m-1)\min_{(i,j)\in\mathcal{A}} c_{ij}$: STOP.
       * $\pi_j = i$.
       * $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.
   - If $\mathcal{S} = \emptyset$: STOP.
   - Otherwise, start again.

Note that this version includes the statement $\pi_j = i$, that keeps memory of the previous nodes in the path. At the end of the algorithm, the vector π is such that π_i is the node preceding node i in the shoretest path, if $i \neq o$ and $\lambda_i \neq \infty$.

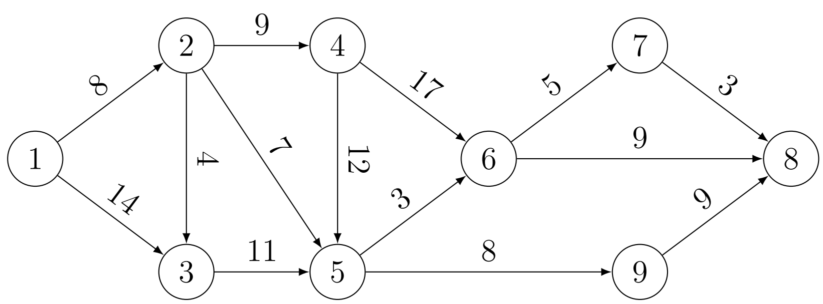
Implement the algorithm. Here are some hints.

1. Implement a class representing a network. In addition to the set of nodes, the set of arcs, and the arc costs, the class should provide the list of successors of each node. Optionally, it is useful to implement a function that plots the network.

2. In Python, the data structure `set` is appropriate to store the nodes to be processed. The statement "Select i in $\mathcal{S}$ and remove it from $\mathcal{S}$" can be performed using the function `pop`.

Use your implementation to identify the shortest paths from node 1 to all other nodes in the following networks. Identify the shortest path tree.

Network 1



Network 2